2022 2nd International Conference on Intelligent Technologies (CONIT)
Karnataka, India. June 24-26, 2022

Hybrid Optimization for DNN Model Compression and Inference Acceleration

Narayan Kulkarni*, Nidhi Singh†, Yamini Joshi‡, Nikhil Hasabi§, S M Meena¶,
Uday Kulkarni‖, Sunil V Gurlahosur**
*, †, ‡,§, ¶, ‖, IEEEauthorrefmark7 *School of Computer Science and Engineering*
KLE Technological University
Hubballi, India
*narayan11kulkarni11@gmail.com

2022 International Conference on Intelligent Technologies (CONIT) | 978-1-6654-8407-7/22/$31.00 ©2022 IEEE | DOI: 10.1109/CONIT55038.2022.9847977

***Abstract*—Deep Neural Networks are known for their applications in the domains like computer vision, natural language processing, speech recognition, pattern recognition etc. Though these models are incredibly powerful they consume a considerable amount of memory bandwidth, storage and other computational resources. These heavy models can be successfully executed on machines with CPU/GPU/TPU support. It becomes difficult for the embedded devices to execute them as they are computationally constrained. In order to ease the deployment of these models onto the embedded devices we need to optimize them. Optimization of the model refers to the decrease in model size without compromising with the performance such as model accuracy, number of flops, and model parameters. We present a hybrid optimisation method to address this problem. Hybrid optimization is a 2-phase technique, pruning followed by quantization. Pruning is the process of eliminating inessential weights and connections in order to reduce the model size. Once the unnecessary parameters are removed, the weights of the remaining parameters are converted into 8-bit integer value and is termed quantization of the model. We verify and validate the performance of this hybrid optimization technique for image classification task on the CIFAR-10 dataset. We performed hybrid optimization process for 3 heavy weight models in this work namely ResNet56, ResNet110 and GoogleNet. On an average, the difference in number of flops and parameters is 40%. The reduction in number of parameters and flops has negligible effect on model performance and the variation in accuracy is less than 2%. Further, the optimized model is deployed on edge devices and embedded platform, NVIDIA Jetson TX2 Module.**

***Index Terms*—Deep Neural Network, Classification, Hybrid Optimization, Pruning, Quan tization, Cifar 10 dataset**

I. INTRODUCTION

Deep Neural Networks in a computing system are networks consisting of numerous nodes, layers and connections that are intended to imitate the working of a human brain. They are used to address various problems of Artificial Intelligence in today's world. We encounter these in various fields like computer vision, natural language processing, speech recognition, etc. Deep Neural Networks is one such neural network with a deeper network consisting of multiple hidden layers and units, hence the name Deep Neural Network [1]. The deployment of these deep learning software has gained a lot of attention because of the increasing demand in their applications [2]. Though these models are incredibly powerful they consume a considerable amount of memory bandwidth, storage and other computational resources. These heavy models can be successfully executed on machines with GPU support. It becomes difficult for the embedded devices to execute them due to their limited computing power. In order to ease the deployment of these models onto the embedded devices we need to optimize them. And this can be done using methods like Pruning [3], Quantization [4], Weight Sharing [5], Log Rank Approximations [6], etc.

In weight Sharing only a shared set of variables are trained for all architectures. The losses of different architectures are computed using these shared weights and these losses are used as estimates of their validation losses. The process of computation speeds up since only a set weights are trained, when compared to earlier methods and the computed validation accuracies have been satisfactory to find good models cheaply. However, this method has been proven to be inconsistent with its results according to recent benchmarks due its poor performance as a substitute for full model-training. Low-rank approximation is a minimization technique used for data compression. It is used for signal processing and machine learning. Here the cost function measures the fit between a given data and the optimization variable, such that the optimized variable has lower rank. It can be used in the process of optimization as we are dealing with a lot of unused weights and parameters which can be considered redundant. Pruning is an iterative process of eliminating inessential weights and connections in order to reduce the model storage size whilst increasing the inference speed. It helps in reducing overfitting. Quantization is a technique using which the tensors are computed and stored at lower bit widths than floating point precision. In a quantized model some or all the operations or computations are done on tensor integers instead of floating point values. This helps in getting a more compact model representation and allows use of high performance vectorized operations on many hardware platforms. This technique saves a lot of inference computation cost without letting it compromise much of inference accuracy and hence is particularly useful at the inference time.

The major contribution of the work presented in this paper is the optimization of deep neural networks using hybrid technique, that is layer wise pruning followed by post training quantization. This is done in order to reduce the model size and number of parameters without compromising the model performance. The three DNNs used here are ResNet-

Authorized licensed use limited to: University of Southern California. Downloaded on March 09,2026 at 00:53:14 UTC from IEEE Xplore. Restrictions apply.

56, ResNet-110 and GoogLeNet. These models are validated and tested for image classification task. On an average, the difference in number of flops and parameters obtained by this hybrid optimization technique is 40%. The reduction in number of parameters and flops has negligible effect on model performance and the variation in accuracy is less than 2%. Further, the optimized models are deployed on edge devices and embedded platforms.

This paper is organized into the following sections. Section 2, focuses on the previous works presented in the field of optimization. Section 3 explains the pipeline followed. It includes data description, model training and testing followed by optimization and evaluation. Last step of the pipeline is comparison and reiteration. The next section includes the results obtained along with the comparison based on model performance, number of flops, parameters and model size. It also mentions the deployment of the models on the embedded devices. The last section concludes the paper along with future scope.

II. Motivation and Related Works

Deep learning has paved its way into various applications in today's world. It has and is coming up with huge neural network models consisting of billions of parameters to achieve state-of-the-art performance. At the same time the network computation requires a massive number of operations in order to perform a single calculation leading to increase in inference time. All these things make it difficult for the model to be deployed on an edge device due to their constrained resources.

The authors of paper [1] deal with the challenges being faced while deploying a deep learning based model on any other device or platform which has limitations for computing power, latency, etc. A huge amount of software applications depend on Deep learning in one or the other way from different domains which includes speech recognition, image processing, disease diagnosis and many more. Deep learning frameworks play an important role while building deep learning models, for example keras, tensorflow, etc. However, the size of the deep learning model has always been a great concern for deploying on any low power device (example mobile phones or any other AI modules) . A DL model which is computationally intensive can easily be executed on PC platforms due to the presence of high GPU support. However deployment of such models on mobile phones or any other low power device becomes a tedious job. Also the changing accuracy and performance have been recorded and in such scenarios. Thus the development and deployment of DL models on low power devices has a lot of challenges which need to be considered. Once the deep learning model gets ready (validated and tested) for real-world usage the deployment process starts and it focuses on how well the deep learning model adapts to a particular platform. Some deep learning frameworks such as TFlite (which is used for Android) and coreML (for iOS) are lightweight and are designed for converting the DL models into the format that is supported by mobile phones. These frameworks are additionally used for optimising the model by quantization. One of the other ways to deal with this problem is to keep the DL software on the server or cloud platforms.

Neural networks are heavy models and are computationally intensive. Thus they need to be optimised before their deployment on embedded devices such as mobile phones which have limited computationally resources and also possess memory constraints. For optimising the heavyweight neural networks it is important to know the contribution of each connection which is present in the network towards the task it is assigned. Since neural networks have a collection of millions and billions of neurons it is essential to understand their role in the network before removing any of the connections in order to optimize the neural network. Once the optimal connection and weights are known, the model or the neural network can be pruned further.

The author in [7] suggests a pipeline for pruning and optimizing the network in 3 steps. Step 1, involves the training of a neural network to understand the contribution of each connection in decision making. Step 2, includes pruning of trivial waits present in the network. Step 3 suggests pruning the model again with the remaining connections so that they can compensate for the lost connections. These three steps are repeatedly performed to get optimal results. This process is also known as iterative pruning. Pruning can also be combined with regularization in an appropriate way to obtain better results. L1 regularization is known to be suited for feature selection and thus can also be used for pruning. It is observed that L2 regularization provides better pruning results.

Pruning can be coupled with appropriate regularization to provide better results. L1 Regularization Equation 1 is best suited for feature selection therefore can be used after pruning but all-in-all L2 Regularization Equation 2 provides better pruning results. Let C denote the cost, the first term of the solution calculates the Loss Function and the second term calculates the Regularization term.

$$C = \sum_{i=0}^{n}(y_i - \sum_{j=0}^{m} x_{ij}w_j)^2 + \sum_{j=0}^{m}|w_j| \tag{1}$$

$$C = \sum_{i=0}^{n}(y_i - \sum_{j=0}^{m} x_{ij}w_j)^2 + \sum_{j=0}^{m} w_j^2 \tag{2}$$

The authors of [3] have focused on reducing the size of convolutional neural networks by pruning filters. It is usually seen that most of the heavyweight CNNs have significant amount of redundancy among different filters and channels. Apart from using the pruned weights from the network, filter pruning is one of the structured ways of pruning without introducing the sparsity value. This paper talks about an acceleration method for convolutional neural networks, where filters are pruned from CNN's which are having smaller effect on the accuracy. This paper concludes that, by reducing the entire filters in the network at once along with their feature maps, the computation cost is significantly reduced

Authorized licensed use limited to: University of Southern California. Downloaded on March 09,2026 at 00:53:14 UTC from IEEE Xplore. Restrictions apply.

The authors of [8] present that deep neural networks are finding their increased applications on the edge device. However,the edge devices have memory and power constraints. Thus there is a need of pressing techniques to optimize the models in order to reduce model size, make fast inference and lower the power consumption. According to [7] one of the simple ways to reduce the complexity of any model is to reduce the precision requirements for the weight and activation. They conclude that activations can be quantized to 8-bits with almost no loss in accuracy [9]. It also concludes that networks with more parameters are more robust to quantization when compared to the networks which have fewer parameters.

Instead of simply eliminating the nonessential filters, the author in [10] suggests that the second order covariance matrix be kept in the pruned model. Preserving this second order information helps in regaining the accuracy of the model faster.New sets of parameters are learnt at each layer by the filtersketch model. The pruned model preserves the second order covariance of the pre-trained model. These new sketched groups of parameters are used as the groundwork when fine tuning the model. This helps in very good recovery of the performance of the model [11]. The filtersketch model aims to maintain the correlation between the parameters of the pruned model and the original model effectively. Hence the pruned model ends up with the second-order covariance of the parameters computed after pre-training of the model. This is done at each layer of the model to obtain the parameters for the pruned model which will then be used to fine tune the pruned model.

III. METHODOLOGY

Deep neural networks are used to address various problems of Artificial Intelligence in today's world. We encounter these in various fields like computer vision, natural language processing, speech recognition, etc. Though these models are incredibly powerful they consume a considerable amount of memory bandwidth, storage and other computational resources. It becomes difficult for the embedded devices to execute them due to their limited computing power. Hence the use of optimization is required to ease the process of deployment.

To achieve the model optimization, this paper is divided into following modules:

1) Dataset Details
2) Model Building and training
3) Hybrid Optimization
4) Model Evaluation
5) Comparision and Reiteration

Figure 1, shows the pipeline of the proposed work. We approach this problem for optimization of the model using hybrid optimization approach.

A. Dataset

The Dataset used in this project is Cifar-10. The dataset consists of 60000 images. There are 10 classes included in this dataset. They are - dogs, airplanes, trucks, cats, horses, deer, frogs, ships, birds, and cars. 50000 of these images are used for training and the remaining 10000 are used for testing. The data set has 60000 coloured images of low resolution 32 x 32. And the test set contains 10000 randomly selected images from each class.

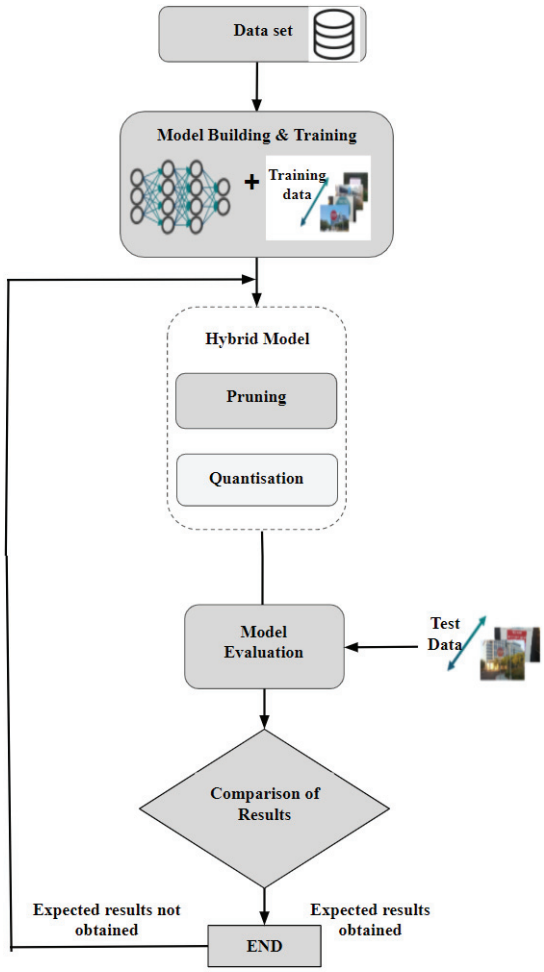

Fig. 1. Methodology.

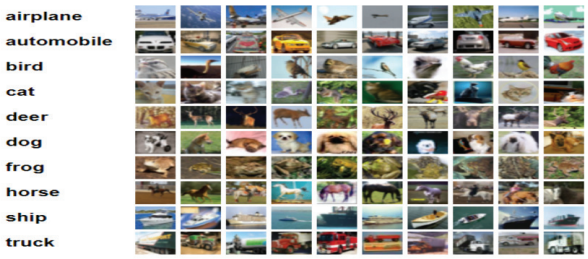

Fig. 2. Sample images from Cifar-10 dataset with 10 samples.

Authorized licensed use limited to: University of Southern California. Downloaded on March 09,2026 at 00:53:14 UTC from IEEE Xplore. Restrictions apply.

B. Model Building and Training

The GoogLeNet ResNet-56 and ResNet-110 models are trained using the 50000 training images of the dataset. GoogLeNet consists of 22 layers with over 6.7 million parameters. It is a variant of Inception Network. GoogLeNet (or Inception V1) was proposed by Google researchers in the research paper [12] called "Going Deeper with Convolutions" . The input to this network is an image of 224x224 resolution with RGB colour channels. All the computational layers use Rectified Linear Unit (ReLU) activation function. It has a softmax layer with the required number of classes (in this case 10 classes) as the output layer.

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

Fig. 3. Layer by layer architecture details of GoogLeNet.

ResNet-56 is a residual network with 56 layers using the core idea of ResNets that is skip connections. In skip connections[13], there are connections between different layers in which the weights of a certain layer are passed on to a different layer by skipping layers in between (as shown in fig 4.4). These skip connections make sure that the neural network is dynamic and trains optimally [14]. ResNet-56 takes an input image of 224x224x3 resolution.And outputs the class to which a given image belongs based on the training data.

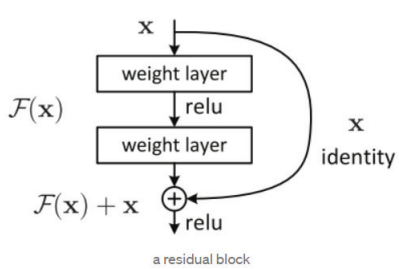

Fig. 4. Residual Block (skip connection).

ResNet-110 is also a residual network with 110 layers. For such deeper residual networks, each residual block has 3 layers. 1x1, 3x3, 1x1 convolutions are the convolution blocks.Finally we have an average pooling layer, which is followed by a fully connected layer. The fully connected layer has 10 neurons (that is the number of classes present in the Cifar-10 dataset). Here in this module the neural network models were trained and the parameters (weights) were learnt and these weights were pruned in the next step.

C. Hybrid Optimization

In this step the model is optimized using hybrid optimization [15]. Hybrid optimization refers to the combination of pruning and quantization. Pruning is the process of removing nodes and connections from a network. It blocks a leaf node thus removing an entire subtree thus ensuring fewer parameters and reduced overfitting. There are two types of pruning: structured pruning and unstructured pruning. In unstructured pruning the weights and connections are eliminated by making them 0. In structured pruning an entire channel or filter is removed as part of pruning. Here groups of weights are eliminated and not just individual nodes. But when compared to unstructured pruning, structured pruning extremely restricts the highest sparsity that can be imposed on a neural network. Hence severely restricting the performance. In the process of pruning inessential weights and connections are eliminated [16]. Thus reducing the model size. The sparsity level set for the GoogLeNet and ResNet models are set while pruning the model. Sparsity here refers to the number of elements in a tensor that are 0. If most elements of a tensor are equal to 0 it is called sparse. Here, new sets of parameters are learnt at each layer by the the model. The pruned model preserves the second order covariance of the pre-trained model. Preserving this second order information helps in regaining the accuracy of the model faster. The model aims to maintain the correlation between the parameters of the pruned model and the original model effectively. Hence the pruned model ends up with the secondorder covariance of the parameters computed after pre-training of the model. As discussed earlier [11] suggests that preserving the second order covariance matrix helps in regaining the accuracy. Since huge accuracy loss would degrade the performance of the model, even if the model is optimized. One of the important aspect for optimisation of the model is not compromising with the performance. Thus, instead of simply eliminating the non essential filters from the model in order to reduce the weight, we preserved the second order covariance and thus the model is capable to learn new sets of parameters at each layer. This work aims to improve the performance of the model at each step along with reducing the size of the model. Quantization is a technique using which the tensors are computed and stored at lower bit widths than floating point precision [17]. In a quantized model some or all the operations or computations are done on tensor integers instead of floating point values. This helps in getting a more compact model representation and allows use of high performance vectorized operations on many hardware platforms. Thus saving a lot of inference computation cost without letting it compromise much of inference accuracy and hence is particularly useful at the inference time.

Training Strategy: The Stochastic Gradient Descent with momentum 0.9 as loss function is used for fine-tuning the model and the batch size is set to 256. The weight decay is set to 5e-3 for CIFAR-10 dataset. The number of epochs is set

Authorized licensed use limited to: University of Southern California. Downloaded on March 09,2026 at 00:53:14 UTC from IEEE Xplore. Restrictions apply.

to 150 with initial learning rate of 0.1, which is then divided by 10 after every 50 epochs. The standard data augmentation techniques provided by the official Pytorch like Random crop and horizontal flip is applied. Performance Metric: Parameter amount and FLOPs (floating-point operations) are used as the metrics, which denotes the storage and computation cost respectively

Algorithm 1 Frequent Direction

Require: Matrix $W^i \in \mathbb{R}^{d_i \times c_i}$, sketched size $\tilde{c}_i$.
Ensure: Sketched matrix $\Omega^i \in \mathbb{R}^{d_i \times c_i}$.
0: $\Omega^i \leftarrow$ all zeros matrix $\in \mathbb{R}^{d_i \times c_i}$
0: **for** each column W^i_j in W^i **do**
0: Insert W^i_j into a zero valued column of Ω^i ,
0: **if** Ω^i has no zero valued columns **then**
0: [U,S,V] = $SVD(\Omega^i)$,
0: $S = \sqrt{max(S^2 - I_{\tilde{c}_i})}$ Identity Matrix $I_{\tilde{c}_i} \in \mathrm{R}^{\tilde{c}_i \times \tilde{c}_i}$
0: $\Omega^i = U\tilde{S}$. at least half columns of Ω^i are zeros
0: **end if**
0: **end for**=0

As shown in Algorithm 1 The Frequent Direction (FD) method, a type of deterministic sketching algorithm is used where in each sample is passed forward only once without iterations, which is extremely efficient.

As shown in Algorithm 1 The Frequent Direction (FD) method, a type of deterministic sketching algorithm is used where in each sample is passed forward only once without iterations, which is extremely efficient.

Algorithm 2 FilterSketch Algorithm

Require: Pre-Trained Model F with filter set $W = \{W^i\}_{i=1}^L$,
Ensure: Pruned Model $\mathcal{F}$ with filter set $\Omega = \{\Omega^i\}_{i=1}^L$ and per-layer pruning rate set $\{p_i\}_{i=1}^L$
0: **for** i= 1 $\leftarrow$ L **do**,
0: Compute the sketched size by $c_i = \lfloor p_i \cdot c_i \rceil$,
0: Obtain Ω^i via sketching W^i by Algorithm.1,
0: Normalize Ω^i via $\ell 2$-norm
0: **end for**
0: Initialize F with the sketched filter set Ω
0: **for** t=1 $\leftarrow$ T **do**
0: Fine-tune the pruned model $\mathcal{F}$
0: **end for**
return $\mathcal{F}$ with the fine-tuned filter set Ω =0

The overall working of layer wise Pruning Technique, from pruning the pre-trained model till fine tuning the pruned model is shown in Algorithm 2.

D. Model Evaluation

The thus obtained model is used to test on the test set of images. The results obtained are compared with the original results. The accuracies are compared to ensure that there is no much loss in accuracy and also the model size, number of parameters and the inference time is reduced. If these conditions are satisfied the process is stopped and the final optimised model is obtained. Else the process is repeated again and is known as iterative pruning.

E. Comparison and Reiteration

Once the optimised model is obtained the results if that model are compared with the baseline model. If the accuracy obtained by the pruned model is less than 2 percent from the baseline model then the optimization process is repeated. This is done till and optimal model with reduced model size and acceptable accuracy is obtained. Once the desirable results are obtained the process stops and we are left with the optimized model ready to be deployed.

IV. Results and Discussion

By this work, it is visualised as more we prune the model, more is the loss in size and less is the accuracy. However, the reduction in the accuracy and performance of the model should be negligible to retain the efficiency of the model.

A. Comparision

For optimising the model we have pruned the model followed by its quantization. Quanti zation converts all the weights of the model to 8- bit integer values. This causes a change in the accuracy and size of the model. However, the number of flops in the model remains the same for pruning and quantization both. Flops are used to describe the number of operations required by a model to perform the specific task. Quantization only converts the weights into 8-bit integer values which does not affect the number of operations by any means. Thus, the number of flops remains unchanged even if the model is quantized. The accuracy of the model gets changed with the pruning and quantization of the model. We have used Accuracy, model size and Flops to compare our results. Accuracy of the model describes the performance of the model. The above obtained results shows that the accuracy of the pruned model decreases slightly which continues to decrease even after hybridisation. However, it is assumed that if the accuracy is not less than 2% then the model performance is affected negligibly. The flops of the model represent the number of operations required to perform a particular task [18]. The obtained results show that the number of flops continues to be the same after pruning and quantization both. The size of the model reduces after pruning and hybridisation. The change in all the parameters signifies the change in the model efficiency and performance. However, the effect on the model performance should be negligible and the model size should change significantly.

The accuracy of the baseline GoogleNet model was 95.05 which changed to 94.23% and 94.10% after pruning and quantization respectively. Similarly, the baseline accuracy of ResNet56 and ResNet110 was 93.26% and 93.50% respectively which changed to 92.54% and 91.96% in case of ResNet56 after pruning and hybridisation respectively and 92.31% and 92.12% in case of ResNet110. This can be observed in the graph of Figure 5.

Authorized licensed use limited to: University of Southern California. Downloaded on March 09,2026 at 00:53:14 UTC from IEEE Xplore. Restrictions apply.

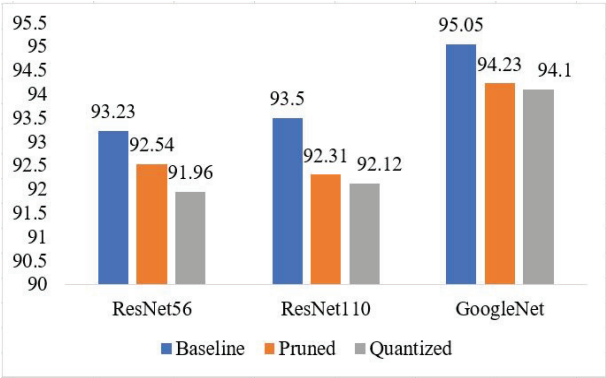

Fig. 5. Comparison of accuracies before and after Optimization.

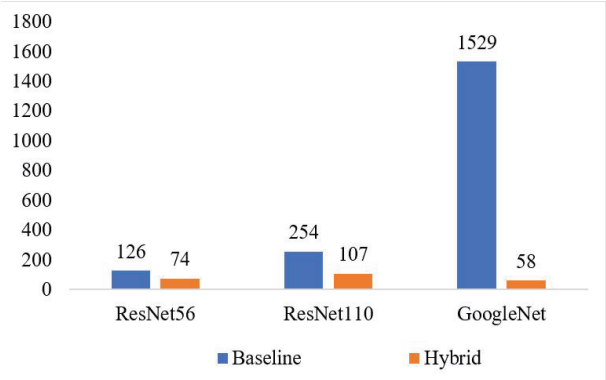

Fig. 6. Comparison of flops before and after Optimization

Before, pruning the number of flops for GoogleNet was 1529 millions which changed to 58 millions. Similarly, for Resnet56 it changed from 126 millions to 74 millions and 254 millions to 10 millions in case of ResNet110. This can be seen in the graph of Figure 6.

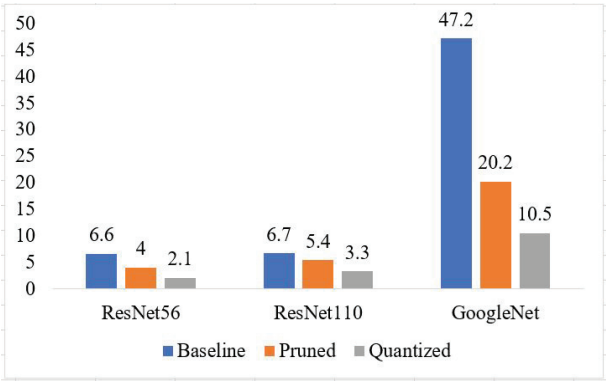

Fig. 7. Comparison of the size of the models before and after Optimization.

Similarly, the size of the baseline GoogleNet, ResNet56 and ResNet100 model was 47.2MB, 6.6MB and 6.7MB respectively which was reduced to 10.5MB, 2.1MB and 3.3MB after hy bridization. In between, the pruned model had size 20.2MB, MB and 5.4MB respectively. The size of the model had a significant change after model optimization. This can be observed in the graph of Figure 7.

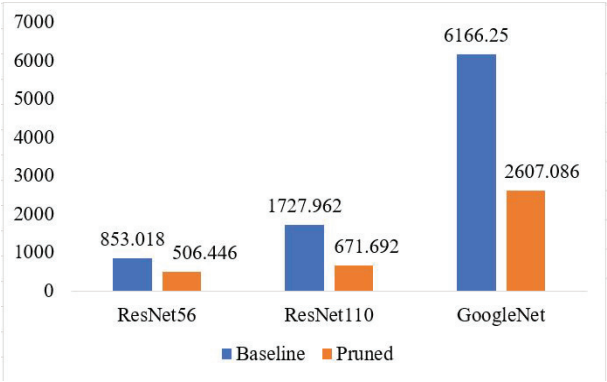

Fig. 8. Comparison of the number of parameters of the models before and after Optimization.

The number of parameters of the baseline GoogleNet, ResNet56 and ResNet110 model was 6166.25, 853.018 and 1727.962 thousands respectively which was reduced to 2607.086, 506.446 and 671.692 thousands after hybridization. The number of parameters had a significant change after model optimization. This can be observed in the graph of Figure 8.

B. Deployment on Jetson TX2

Jetson TX2 is a developer kit which is mainly used for visualising the computed design of the model. It comes with a pre-installed Linux environment, a set of commonly used APIs and is supported by NVIDIA's complete development tool chain. It is an embedded device which is used in this project to compare the model performance before and after optimization. As discussed, the process of optimisation in this work includes pruning and quantization which is commonly known as hybridisation. The Jetson TX2 module comprises a wide variety of hardware interface which ideally builds this module a platform to visualise computationally heavy models with low power usage [19]. The developer kit contains a reference carrier board and module, an AC adapter, power cord, a USB Micro B to USB A adapter, a copy of the Quick Start Guide, Safety Guide, and two Antennas for Wi-Fi. All these components are connected appropriately to make the Jetson TX2 module work for huge and heavy models. This module is being widely used these days in the field of Artificial Intelligence due to its capability of delivering the performance and power efficiency which is needed by many of the latest computing models. It is built with the revolutionary NVIDIA Maxwell™ architecture with 256 CUDA cores delivering over 1 TeraFLOPs of performance. 64-bit CPUs, 4K video encode and decode capabilities, and a camera interface capable of 1400 MPix/s make this the best system for embedded deep learning, computer vision, graphics, and GPU computing. Since, this module has all the required capabilities and deliverables required for our project, we chose this module to compare the model performance (baseline as well as pruned and hybridised model) with respect to its working on the

Authorized licensed use limited to: University of Southern California. Downloaded on March 09,2026 at 00:53:14 UTC from IEEE Xplore. Restrictions apply.

embedded and edge device. Since, the module comes with pre installed Linux features, the interface also looks similar to it. It has the capability to deliver all the functions which are available in Linux operating system. This pre installation of Linux makes the module to be used easily by the developer.

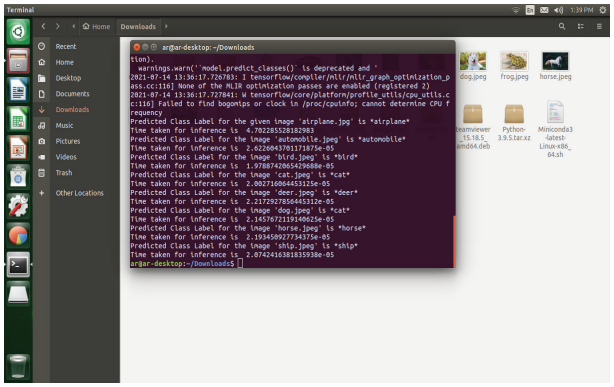

Fig. 9. Model prediction using NVIDIA Jetson TX2 module

Figure 9 represents the working of our model on Jetson module. There is a terminal which is similar to the one present in the Linux environment. The predicted class labels for each of the selected image are getting printed on the terminal. Since, this work focuses on the optimization of the model, the inference time i.e. the time taken by the model for decision making plays an important role. Thus, we have also displayed the inference time taken by the model for each of the selected image.

C. Comparison of Inference time between Jetson TX2 and Android Device

For comparison of performance of hybrid optimized model, we deployed the model on NVIDIA Jetson TX2 and also on Android Device. On an average, The NVIDIA Jetson TX2 Module takes 20 microseconds and android device takes 65000 microseconds or 65 milliseconds.

The average time taken for inference for predicting class labels of given 10 test images can be seen in Table I.

NVIDIA Jetson TX2	Android Device
20	65000

TABLE I
AVERAGE INFERENCE TIME IN MICROSECONDS

D. Model Evaluation

The thus obtained model is used to test on the test set of images. The results obtained are compared with the original results. The accuracies are compared to ensure that there is no much loss in accuracy and also the model size, number of parameters and the inference time is reduced. If these conditions are satisfied the process is stopped and the final optimised model is obtained. Else the process is repeated again and is known as iterative pruning.

E. Comparison and Reiteration

Once the optimised model is obtained the results if that model are compared with the baseline model. If the accuracy obtained by the pruned model is less than 2 percent from the baseline model then the optimization process is repeated. This is done till and optimal model with reduced model size and acceptable accuracy is obtained. Once the desirable results are obtained the process stops and we are left with the optimized model ready to be deployed.

V. CONCLUSION AND FUTURE SCOPE

The method proposed in this work helps in compressing the deep neural network model and reducing the model size and the inference time. The results mentioned in section 5, show how the methods used have been successful in significantly reducing the number of weights and parameters in turn reducing the model size. So that this lightweight model can be easily deployed on embedded devices with less computational resources with ease. The optimization algorithm can be used to reduce the complexity of various deep neural network models to make them lightweight and easy to deploy. This optimization will also make sure that the models do not overfit. During the process of pruning the weights and parameters that are eliminated depend on different types of pruning, like layer wise pruning, channel wise pruning, etc. Here in this project we have used layer pruning. [20] In future we can try with other kinds of pruning methods in order to reach better results. So that the model size can be reduced efficiently without compromising the accuracy.

REFERENCES

[1] Alberto Prieto, Beatriz Prieto, Eva Martinez Ortigosa, Eduardo Ros, Francisco Pelayo, Julio Ortega, and Ignacio Rojas. Neural networks: An overview of early research, current frameworks and new challenges. Neurocomputing, 214:242–268, 2016.

[2] Zhenpeng Yanbin Cao Yuanqiang Liu Haoyu Wang Tao Xie Chen and Xuanzhe Liu. A comprehensive study on challenges in deploying deep learning based software. Proceed ings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2020.

[3] Asim Kadav Igor Durdanovic Hanan Samet Li Hao and Hans Peter Graf. Pruning filters for efficient convnets.

[4] Vamsi Sripathi Deepthi Karkada Vivek Menon Sun Choi Kushal Datta Bhandare Aish warya and Vikram Saletore. Efficient 8- bit quantization of transformer neural machine language translation model. 2019.

[5] Melody Guan Barret Zoph Quoc Le Pham Hieu and Jeff Dean. Efficient neural archi tecture search via parameters sharing. International Conference on Machine Learning, 2018.

[6] Feiping Nie Rong Wang Hu Zhanxuan and Xuelong Li. Low rank regularization: A review. International Conference on Machine Learning, 2020.

[7] Han Song Jeff Pool John Tran and William J. Dally. Learning both weights and connec tions for efficient neural networks. arXiv preprint arXiv:1506.02626, 2015.

[8] Wu Rih-Teng Ankush Singla Mohammad R. Jahanshahi Elisa Bertino Bong Jun Ko and Dinesh Verma. Pruning deep convolutional neural networks for efficient edge computing in condition assessment of infrastructures. Computer-Aided Civil and Infrastructure Engineering 34, no. 9: 774-789, 2019.

Authorized licensed use limited to: University of Southern California. Downloaded on March 09,2026 at 00:53:14 UTC from IEEE Xplore. Restrictions apply.

[9] Tang Wenting Xingxing Wei Bo Li. Automated model compression by jointly applied pruning and quantization. arXiv preprint arXiv:2011.06231, 2020.
[10] Lin Mingbao Liujuan Cao Shaojie Li Qixiang Ye Yonghong Tian Jianzhuang Liu Qi Tian and Rongrong Ji. Filter sketch for network pruning. IEEE Transactions on Neural Networks and Learning Systems, 2021
[11] He Yihui Ji Lin Zhijian Liu Hanrui Wang Li-Jia LiSong Han. Amc: Automl for model compression and acceleration on mobile devices. In Proceedings of the European Confer ence on Computer Vision (ECCV), 2018.
[12] Szegedy Christian Wei Liu Yangqing Jia Pierre Sermanet Scott Reed Dragomir Anguelov Dumitru Erhan Vincent Vanhoucke and Andrew Rabinovich. Going deeper with convolu tions. In Proceedings of the IEEE conference on computer vision and pattern recognition, 2015.
[13] He Kaiming Xiangyu Zhang Shaoqing Ren Jian Sun. In proceedings of the ieee conference on computer vision and pattern recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770-778, 2016.
[14] Stanisław Jastrzebski, Devansh Arpit, Nicolas Ballas, Vikas Verma, Tong Che, and Yoshua Bengio. Residual connections encourage iterative inference. arXiv preprint arXiv:1710.04773, 2017.
[15] Shiliang Sun, Zehui Cao, Han Zhu, and Jing Zhao. A survey of optimization methods from a machine learning perspective. IEEE transactions on cybernetics, 50(8):3668–3681, 2019.
[16] Guiying Li, Chao Qian, Chunhui Jiang, Xiaofen Lu, and Ke Tang. Optimization based layer-wise magnitude-based pruning for dnn compression. In IJCAI, pages 2383–2389, 2018.
[17] James Stokes, Javier Robledo Moreno, Eftychios A Pnevmatikakis, and Giuseppe Car leo. Phases of two-dimensional spinless lattice fermions with first-quantized deep neural network quantum states. Physical Review B, 102(20):205122, 2020.
[18] Raphael Tang, Ashutosh Adhikari, and Jimmy Lin. Flops as a direct optimization objec tive for learning sparse neural networks. arXiv preprint arXiv:1811.03060, 2018.
[19] Sparsh Mittal. A survey on optimized implementation of deep learning models on the nvidia jetson platform. Journal of Systems Architecture, 97:428–442, 2019.
[20] Risto Miikkulainen, Jason Liang, Elliot Meyerson, Aditya Rawal, Daniel Fink, Olivier Francon, Bala Raju, Hormoz Shahrzad, Arshak Navruzyan, Nigel Duffy, et al. Evolving deep neural networks. In Artificial intelligence in the age of neural networks and brain computing, pages 293–312. Elsevier, 2019.

Authorized licensed use limited to: University of Southern California. Downloaded on March 09,2026 at 00:53:14 UTC from IEEE Xplore. Restrictions apply.